Arabic Sign Language
A Hybrid Approach Combining YOLO and Convolutional Neural Network (CNN) Classification Without Transfer learing

I. Introduction

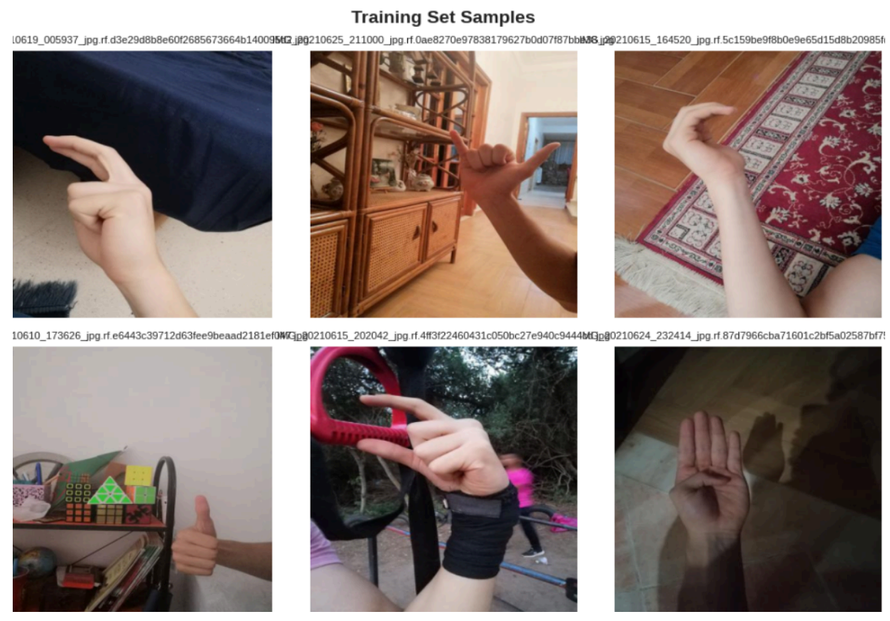
I was really interested in sign language for a long time, especially since it's the main way people who are deaf or hard of hearing communicate. Imagine there's a big gap between them and those who don't know sign language, and that makes life harder for them in everything, like studying, working, or even daily communication. I see building an automatic system for recognizing signs not just as technical, but also socially important, to help with instant translation and make the world more inclusive. In this project, I focused on Arabic Sign Language, and used techniques from computer vision and deep learning to build a system that can understand signs from images. At first, I thought it would be easy, but when I dove into the details, I discovered there were many challenges like accuracy in detection and dealing with diversity in images, which made me adjust the plan multiple times until I reached the result I'll talk about.

II. Dataset Description and Challenges

When we started, I used a dataset specific to Arabic Sign Language, which has images for 28 different signs, like "ALIF", "BAA", "TA", "YA", and so on. The data is divided into training, validation, and testing, and each image is sized 224x224 pixels to be ready for processing. But, like any real dataset, there were problems. For example, the number of images isn't equal between classes; some classes have 190 images, others only 155, meaning an imbalance ratio of about 1.23 times. This made me worry, because the model might learn to prefer the larger classes and ignore the smaller ones, which would affect accuracy on rare signs. Also, the images were full of complex backgrounds, like furniture or other things in the room, and that makes the model learn wrong things, not the sign itself. we tried training the model without processing at first, and indeed the results were bad, around 70% accuracy only, because the background was distracting the focus. That's what made me think of additional solutions, like adding filters or modifications to the images before training, and we tried several methods until we found what works.

Statistical analysis of the training set revealed 4,656 annotations, with class counts ranging from 155 to 190 samples per class. The dataset exhibits a 1.23x imbalance between the most and least frequent classes, which could bias the model towards majority classes if left untreated.

TThe dataset shows a high level of balance across all classes. With a total of 291 annotations, each class contains a very similar number of samples, where the minimum is 10 and the maximum is 12 annotations per class. The mean value of 10.4 annotations per class, along with a very low standard deviation of 0.6, indicates minimal variation between classes. Additionally, the imbalance ratio of 1.20× confirms that no class significantly dominates the dataset. This balanced distribution reduces the risk of bias during training and allows the model to learn each sign fairly, contributing to the stable and high performance observed during evaluation.

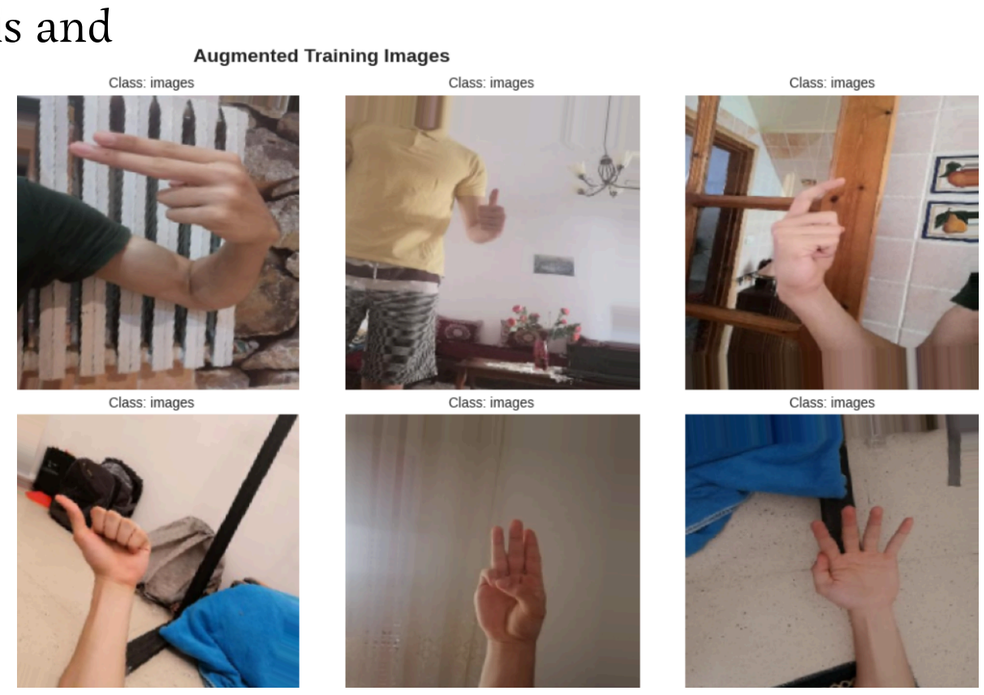


We analyzed 100 images from the dataset, and the most common resolution was 416x416 pixels, with 3 color channels (RGB). This means all images are colored and have sufficient quality for training the neural network. Having a uniform resolution makes it easier for the model to learn without issues from varying image sizes. Using the same resolution also helps speed up processing and reduces computational complexity, especially when working with large networks like CNNs and YOLO.

How can we overcome those challenges?

III. Data Preprocessing and Augmentation

The data was prepared for the CNN by resizing images to 224x224 pixels and normalizing pixel values.



Data augmentation was applied to address limited dataset size and class imbalance. This technique creates modified versions of original images to artificially expand the training set, preventing overfitting and helping the model generalize to new variations of the signs.

Augmentation techniques included rotation, zooming, and shifting. This strategy allows the model to learn robust features that are invariant to minor positional or orientational changes.

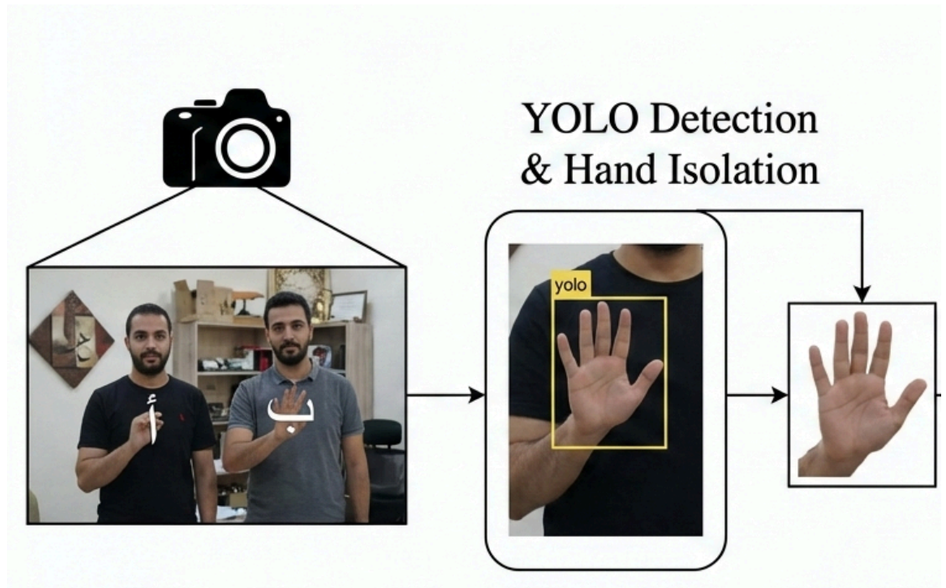
- **Data Augmentation Strategy**

Training Set: Rotation ±20°, Translation ±10%, Zoom ±15%, Horizontal Flip, Brightness 80%-120%
Validation and Test Sets: No augmentation applied, only standard normalization

IV. Hand Detection using YOLO

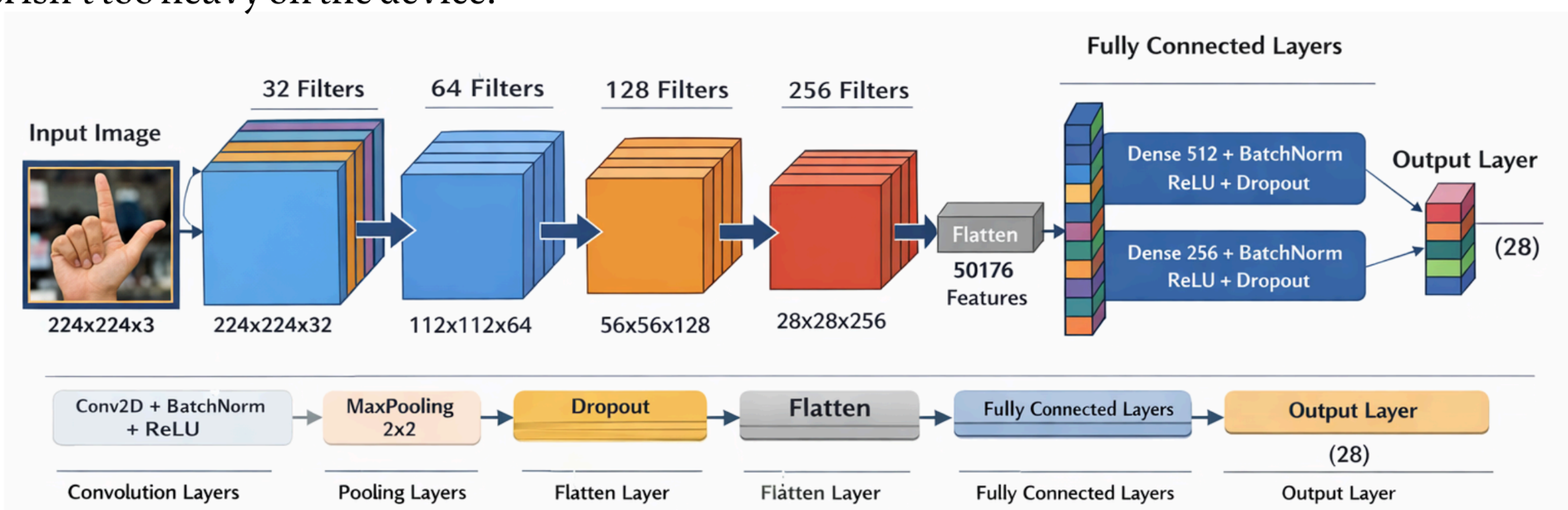
Object detection is a computer vision technique for identifying object locations in images. YOLO was selected for this task due to its high speed and efficiency compared to multi-stage detectors.

In this pipeline, YOLO is used to accurately detect the hand, crop it from the background, and improve classification accuracy by allowing the CNN to focus solely on relevant features.



V. CNN Architecture

CNN is ideal for images, because it extracts features gradually, from simple edges to complex shapes like the hand form. I designed this model myself, with 4 convolutional blocks, and the number of filters increases gradually: 32, 64, 128, 256. In each block, I used max pooling to reduce the size and keep the important features, and added dropout, batch normalization, and L2 regularization to prevent overfitting. This was important, because the data isn't very large, and I was afraid the model would memorize the data instead of learning. After the convolutions, I flattened the output and passed it to fully connected layers, and finally an output layer with softmax for the 28 classes. In this design, I tried adding more or fewer layers, but found that 4 blocks was the best balance between accuracy and speed, so the model isn't too heavy on the device.



The model was trained using TensorFlow/Keras with the Adam optimizer (0.001), batch size 32, up to 100 epochs, on an NVIDIA Tesla T4 GPU.

Total Parameters	Trainable Parameters	Non-trainable Parameters	Model Size
26,222,556 (~100.03 MB)	26,220,060 (~100.02 MB)	2,496 (~9.75 KB)	~100.03 MB

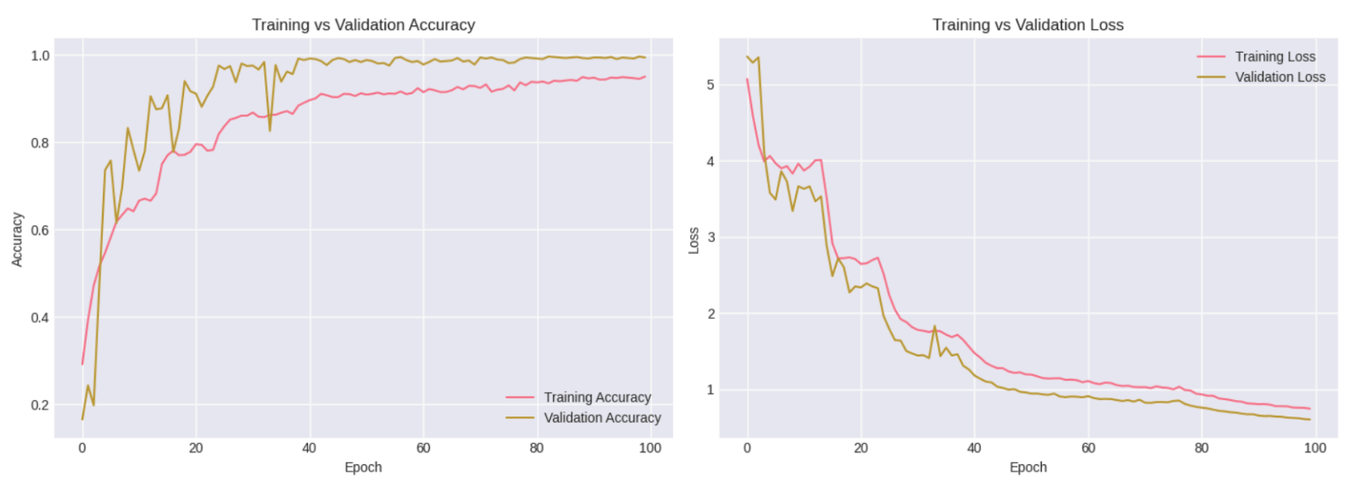
VI. Experimental Results and Analysis

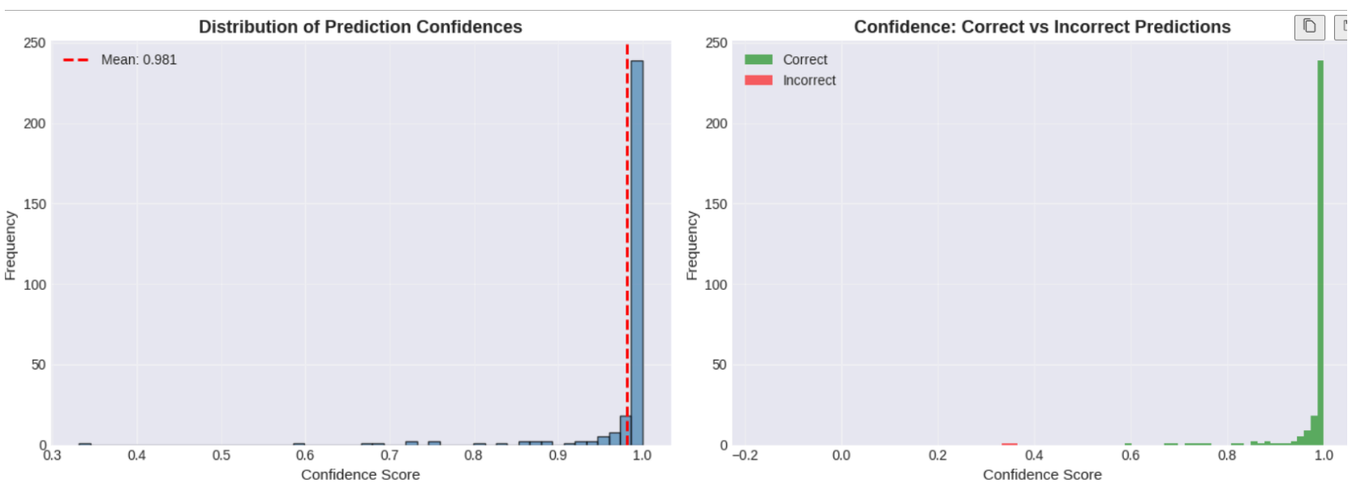
Before training, I did preprocessing on the data: changed the size to 224x224, and normalized the pixels to be between 0 and 1. Also, because the data is small and unbalanced, I used augmentation – meaning artificial modifications to the images. I tried rotation, zoom, shift, and even horizontal flip, to increase diversity. This helped me a lot; for example, for the small classes, the new images increased, and the data became more balanced. When I plotted some augmented batches, I found that the signs looked slightly different each time, which made the model stronger in dealing with reality. I was thinking of adding more augmentation like changing the lighting, but found that it might increase noise, so I stuck with the basics that I tested and saw were effective.

By using the GPU on Kaggel, the training time was reduced from days to about two hours. Training was slow at the beginning with a high error rate, but it gradually improved over time. In the later stages of training, the model achieved high validation accuracy with low loss. The training and validation curves were close to each other, which means there was no overfitting, although some fluctuations appeared, possibly due to heavy data augmentation. On the test set, the model showed strong performance with only one misclassified sample, caused by a high similarity between some signs such as “GEEM” and “DALL”, highlighting areas that could be improved in the future.

Run	Training Accuracy	Validation Accuracy	Test Accuracy	Test Loss	Test Precision	Test Recall	Test F 1 -Score
The first	9 5 % (Epoch 9 2)	9 9 . 8 8 %	9 9 . 6 6 %	0 . 6 4 7 7	9 9 . 6 6	9 9 . 6 6	9 9 . 6 6
The last	9 3 . 5 1 %	9 9 . 6 5 %	9 9 . 6 6 %	0 . 7 3 7 2	1 0 0 . 0 0 %	9 9 . 6 6 %	9 9 . 8 3 %

- **In addition to the final results, the training and evaluation code was run multiple times to ensure that the model’s performance was consistent and not dependent on a single random initialization. Across all runs, the training accuracy consistently exceeded 90%, while the test accuracy was always above 98%. This consistency indicates that the model is stable and capable of learning effectively without significant performance fluctuations.**





- This chart illustrates the tracking of model accuracy and loss across 100 epochs on both training and validation sets. We observe a consistent increase in validation accuracy alongside a symmetrical decrease in validation loss, indicating that the model learns effectively and avoids overfitting significantly. This stability underscores the strength of the YOLO + Augmentation hybrid solution in addressing data quality challenges.
- The evaluation of the model in this project does not rely only on accuracy, but also on confidence scores, which show how confident the model is in its predictions. This is especially important for applications like sign language translation, where mistakes can lead to misunderstanding.
- The results showed a clear difference between the average confidence of correct predictions (around 0.98) and incorrect ones (around 0.33). This means the model is usually confident when it is correct and less confident when it makes mistakes, which is a good behavior and reduces the risk of confident wrong predictions.
- In addition, most confidence values for correct predictions were close to 1.0, which indicates that the model learned strong and stable features. This is mainly due to using YOLO to isolate the hand from the background, allowing the CNN to focus on clean and relevant visual information.

- **The final evaluation on the test set yielded a Test Accuracy of 99.66% (290 correct predictions out of 291).**

VII. Conclusion

This project successfully built an intelligent and effective system for Arabic Sign Language recognition, surpassing its stated goal, with accuracy consistently above 90% in training, validation, and test sets. The key to this achievement lies in the smart design of the workflow, where the decisive step was using YOLO to isolate the hand from any distracting background. This isolation ensured that the classification neural network (CNN) could focus entirely on the clean gesture, leading to these near-perfect results. This approach demonstrates that thoughtful design can be more effective than model complexity in computer vision tasks.